HEARTLAND AI SYMPOSIUM · WORKSHOP B3

Build a RAG Retrieval Agent

Grady Burns and **Hayden Thatcher**

Wednesday, November 11 · K-State Student Union



What happens when a chatbot doesn’t have our documents?

MEMORY

What it learned in training

- Fixed at a cutoff date
- Mostly public text
- Patterns from what it read

Has never seen your files

CONTEXT WINDOW

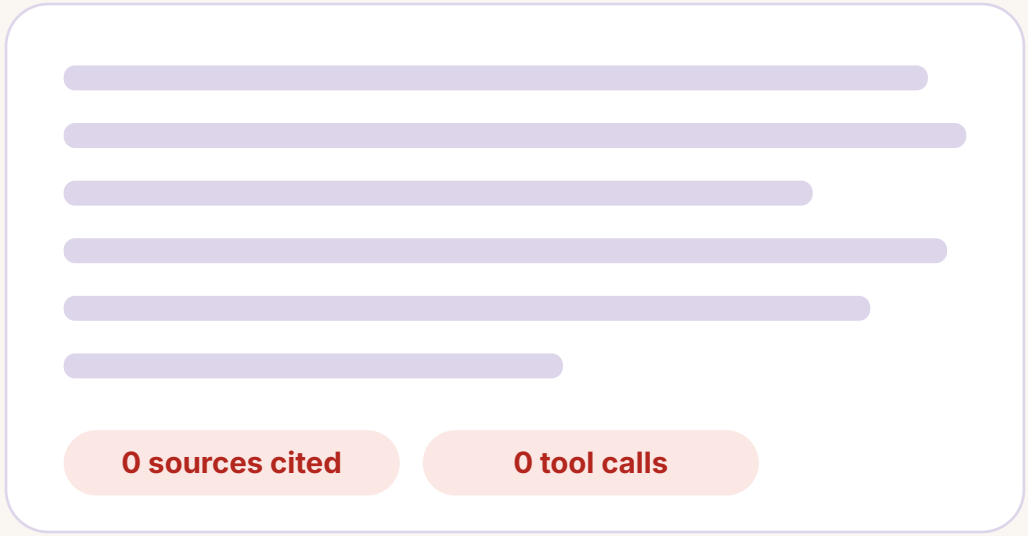
What it’s handed right now

- Its instructions
- Your question
- Anything a tool returns

The only place your documents can be

NO DOCUMENTS CONNECTED

What does the U.S. federal government officially call artificial intelligence now?



Answered from memory, with no source.

Seems plausible,

We have no way to check if it is.

It was right until six weeks ago.

Official U.S. rules and guidance on AI

62

documents

1,847

pages

2019–2026

February 2019 to November 5, 2026

REALISTIC QUESTIONS

- How many grant applications can one researcher submit to NIH in a year?
- Can I use ChatGPT to help me review a grant application?
- What must agencies do before using rights-impacting AI?

Answer from evidence you can identify and be able to check.



START THIS NOW

Send one prompt to any agent on your laptop

SEND THIS TO YOUR AGENT: CODEX, CLAUDE CODE, CURSOR, COPILOT, GEMINI CLI, OPENCODE...

Set up https://github.com/Cambrian-Systems/RAG-Symposium on this computer by following the AGENTS.md file in it. Tell me when it's done and what to do next.

Say yes when it asks to run commands. It downloads the project, loads 1,847 pages and connects the search tools.

No agent yet? We can provide one: install OpenCode (opencode.ai/desktop or USB), /connect, paste the key on your table sheet.

Stuck? Pair with a neighbour. USB sticks and the by-hand commands are on your table.

Browser chatbots can't do this. The agent has to run commands on your laptop.

RETRIEVAL-AUGMENTED GENERATION

Search first, then answer from what was found

R **Retrieve**

Search the documents for passages about the question.

→

A **Augment**

Put those passages into the model's context window.

→

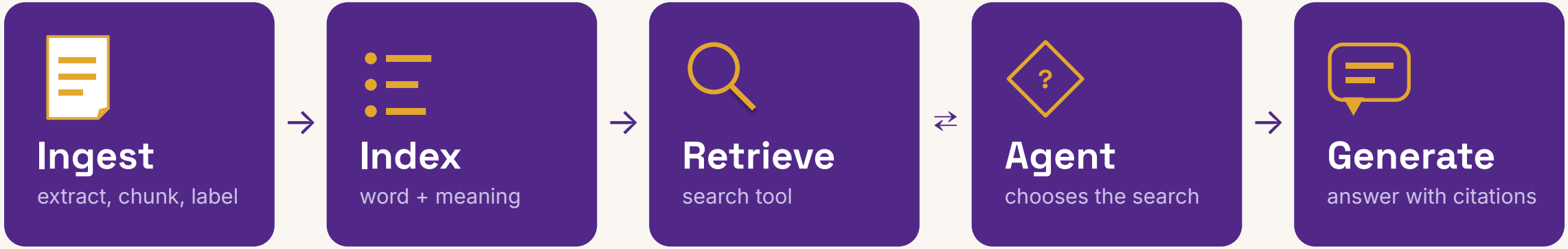
G **Generate**

The model writes its answer from that evidence, and cites it.

RAG is a context pipeline.

Every stage decides what the model sees.

Five stages, and a way to check them



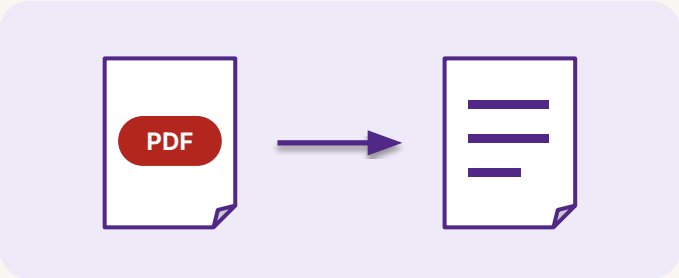
Evaluate a fixed set of questions with known answers, run after every change

FOLLOW ONE QUESTION THROUGH IT

`How many grant applications can one researcher submit to NIH in a year?`

The NIH notice becomes two pages, each labeled July 2025 and in force	Each page is indexed by its words and its meaning	Both pages of the notice come back first	The model writes its own search	“Six per calendar year,” citing page 2

Ingestion changes the documents, not the model



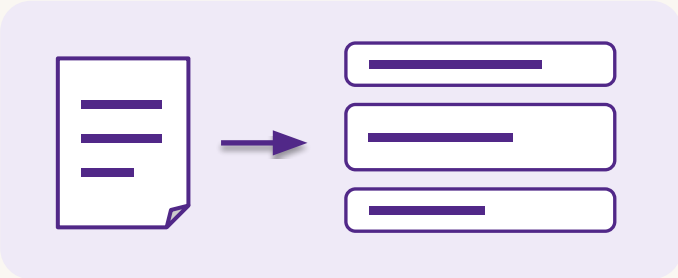
Extract

Turn each PDF into text, once, ahead of time.



Keep the labels

Title, source, page, date and status stay with every piece, so an answer can cite a page and check a date.



Chunk

Split into searchable pieces.

No chunk size is right for every question

By page

what you have today

1,847 pieces

- ✓ A citation points to a page you can open and read.
- ✗ A table or a thought can be split across two pages.

By window

4,214 pieces

- ✓ About 1,500 characters each, so matches are tighter.
- ✗ A piece can lose the heading that explains it.

Whole document

62 pieces

- ✓ Keeps all its context.
- ✗ Hands the model up to 127 pages to find one paragraph.

Chunk size changes which document search puts first

Rank of the right document (BM25)	By page	By window	Whole document
How many grant applications can one researcher submit to NIH in a year?	1st	1st	1st
Can federal education grant money pay for a chatbot that advises students?	1st	2nd	5th
What is the Presidential AI Challenge for students and teachers?	3rd	1st	1st

Index each page by its words and its meaning

Word index

Which pages contain which words, like the index at the back of a book.

`confabulation`  rare: counts a lot

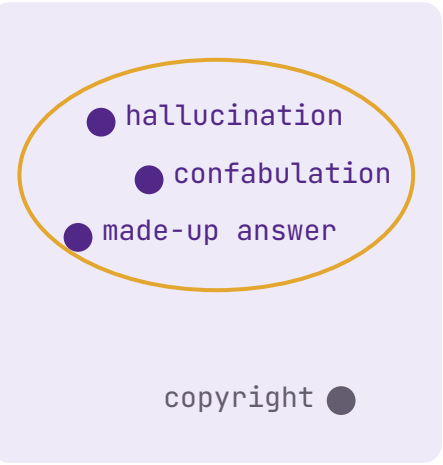
`the` everywhere: counts nothing

The standard scoring is called **BM25**.

Meaning index

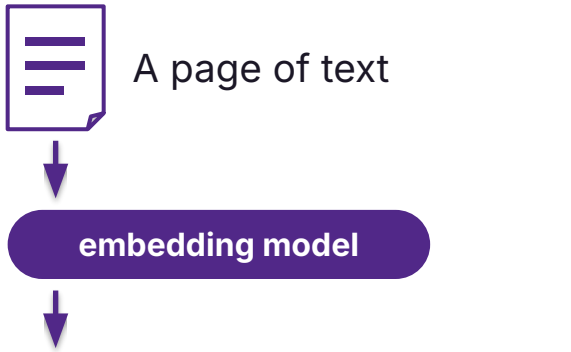
An **embedding model** turns each passage into a list of numbers.

Passages about related things get similar numbers.



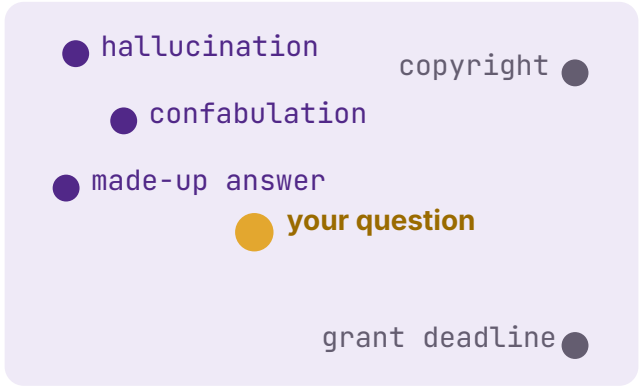
Embeddings find pages with a similar meaning

HOW A PAGE GETS ITS NUMBERS



256 numbers for every page

SIMILAR MEANING, NEARBY NUMBERS



Pages nearest your question come back first. (A sketch.)

Ours is tiny

A 30 MB model that runs on your laptop. Nothing leaves it.

Bigger ones exist

Better at meaning, but slower, often on someone else's server.

Blind spots

IDs like M-24-10, exact numbers, and words newer than the model.

All four ways to rank a page are guesses

Keyword count

Counts your words on the page.

Every word counts the same: “the” matters as much as “M-24-10.”

the simplest baseline

BM25

Same, but rare words count for more.

The default ranking in most search engines.

where you start today

Embeddings

Compares the question’s numbers with each page’s.

The closest meaning wins, even with no words in common.

half of hybrid

Hybrid

Runs BM25 and embeddings, then merges the two lists.

Pages high on either list rank high.

the one to try first

Exact search finds IDs; semantic finds synonyms

Exact-word search

GOOD AT

- IDs like M-24-10
- Names and rare terms
- Numbers

MISSES

- Different words for the same thing

Semantic search

GOOD AT

- Paraphrases
- Synonyms

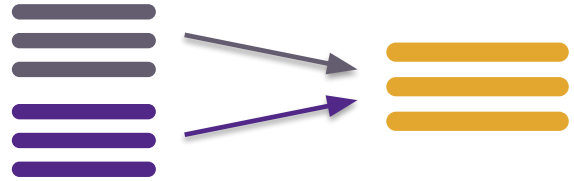
MISSES

- IDs and exact numbers
- Terms the model never learned

Hybrid

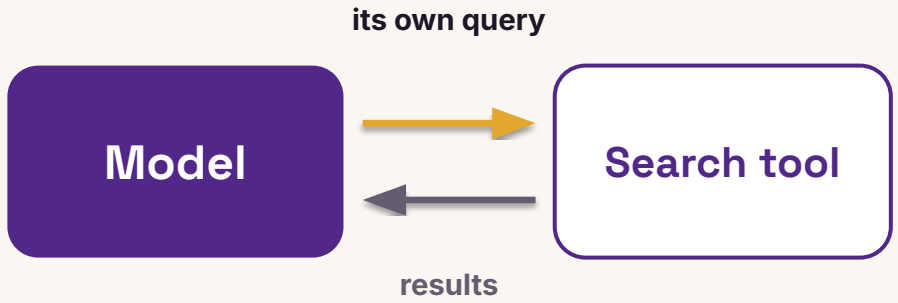
DOES

- Runs both
- Merges the two rankings



Agent: the model decides what to look up

1. The app tells the model which tools it has, and what each one is for.
2. The model decides to search, and writes its own search query.
3. The app runs the search and hands back the results.
4. The model reads them, searches again or opens a full document, then answers.



The search isn’t your question: the model rewrote it. It can also skip the tool.

Generate: answer from the evidence

A GROUNDED ANSWER

NIH accepts at most six applications per researcher each calendar year.

[NIH-NOT-OD-25-132-p2] Jul 2025 · in force

Supporting Fairness and Originality in NIH Research Applications, National Institutes of Health

1 **Ground it**

Answer from the retrieved text, not from memory.

2 **Cite it**

Every claim points to a page you can open.

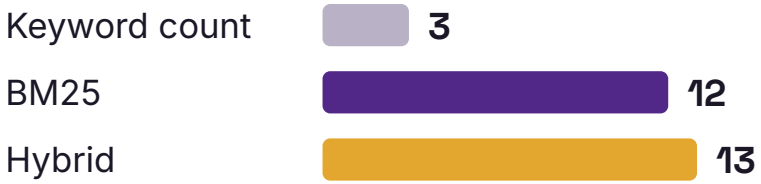
3 **Admit gaps**

Say when the collection can’t answer, and when the source is old.

An eval is a list of questions with known answers

A QUESTION, WORDED THE WAY A PERSON ASKS IT	WHAT SHOULD COME BACK	WHAT BM25 SEARCH DID
What are the four core functions of the NIST AI Risk Management Framework?	NIST-AI-100-1, page 8	✓ Came back first
How does federal law define artificial intelligence?	USC-15-CH119, page 1	✗ Not in the top five
What does the Kansas Board of Regents require of faculty who use AI to grade?	no answer in these documents	✓ Warned: not in the data

RIGHT DOCUMENT FIRST, OUT OF OUR 22 QUESTIONS



- Start with 20 to 50 questions (Anthropic's advice).
- Rerun all of them after every change, and compare.
- BM25 and hybrid are one question apart, which is noise.

Start with questions you've already asked

WHAT YOU'VE TYPED INTO SEARCH	WHAT SHOULD HAVE COME BACK
can nih reviewers use chatgpt	NIH-NOT-OD-23-149, page 1
do i have to tell nsf i used ai	NSF-GAI-MERIT-REVIEW-2023, pages 1–2
k-state ai policy for coursework	no answer in these documents

> Before you have an eval, every question you put to search is already a test. Save them.

The ones that came back wrong are the most useful. Anthropic: start from real failures.

OpenAI's advice: log every question, then mine the log for test cases.

Include a few that should get "no answer," or you only improve one side.

Generated questions run longer than real ones

REAL: WHAT A PERSON TYPES

how many nih grants can i submit

6.8 words on average

GENERATED: WHAT A MODEL WRITES FROM THE DOCUMENT

According to NIH notice NOT-OD-25-132, how many applications may one principal investigator submit per calendar year?

15.7 words on average

Averages from a 2026 study of a university information system: 322 real student questions, 1,851 generated ones (arXiv 2609.14579). The two example questions are ours.

- Generated ones cover every document evenly. Real ones cluster on a few topics, and some have no answer.
- Setups that scored well on generated questions dropped on real ones. Tuning on them picked a slower search.
- Use generated questions to check coverage, and real ones to decide.
- If an agent writes them, say what to vary: who’s asking, the wording, answerable or not, current or replaced.

Ingest › Index › Retrieve › Agent › Generate

Evaluate

Let an agent do the eval work, and spot-check it

AN AGENT CAN DO

1. Pull the questions you've asked out of your session history
2. Find the page that answers each one, or mark it "no answer"
3. Rerun the whole list after every change
4. Sort the misses: not in the collection, buried lower down, or out of date
5. Turn each miss into a new test question

YOU STILL CHECK

- ✓ Spot-check a sample of the answer key. You don't need to open every page.
- ✓ Look into any question that never passes. It's usually a broken question.
- ✓ Read a few of the agent's evals yourself, and compare its grading with your own.

To your agent: **"Add these questions to the evals and run them. Show me the misses."**

Sources: Anthropic, "Demystifying evals for AI agents"; OpenAI, "Evaluation best practices"; Hamel Husain, AI evals FAQ; arXiv 2609.14579 (CIKM 2026).

What’s in the project your agent installed

INGEST

Labeled documents

Every page keeps its title, source, page number, date and status.

INGEST

Three chunk sizes

By page, by window, or the whole document.

INDEX AND RETRIEVE

Three kinds of search

Keyword count, BM25 and hybrid.

AGENT

Three agent tools

Report what’s loaded, find passages with citations, read the page around a result.

RETRIEVE

Missing-word warning

Search says so when a word from your question appears in no result.

EVALUATE

rag-eval

Scores search on a list of questions with known answers.

CHECKPOINT 1

Is your data loaded?

1 Data loaded

data_status reports 1,847 records

→

2 Tools connected

Your agent lists the three tools

→

3 Answer supported

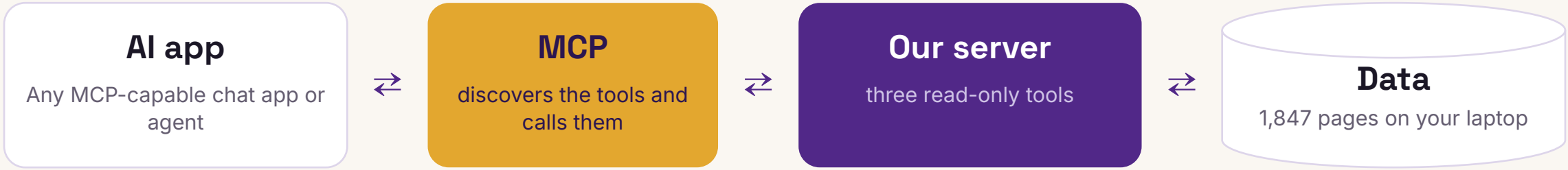
The cited page actually says it

`Did setup finish? How many records are loaded?`

Fallback: run `uv run rag-check` and look for **1,847 records**.

MODEL CONTEXT PROTOCOL

MCP: a standard plug between AI apps and tools



`data_status`	Is the data loaded? How many records? Which search?
`search_documents`	Find passages; returns excerpts with citations
`get_document`	Read the full text around a result

There are thousands of MCP servers: files, email, calendars, databases, GitHub, Slack. Companies publish them; anyone can write one.

Where a chatbot’s tools come from

ON ITS OWN

The model

No tools of its own. It reads and writes text, and can ask for a tool.

The app runs the tool.

BUILT IN

The app

ChatGPT, Claude, Gemini, Copilot: web search, running code, reading your files.

Coding agents: edit files and run commands. That’s how yours installed the project.

Chosen by the app’s maker

ADDED THROUGH MCP

You

Tools the app didn’t ship with.

```
data_status
search_documents
get_document
```

Chosen by you

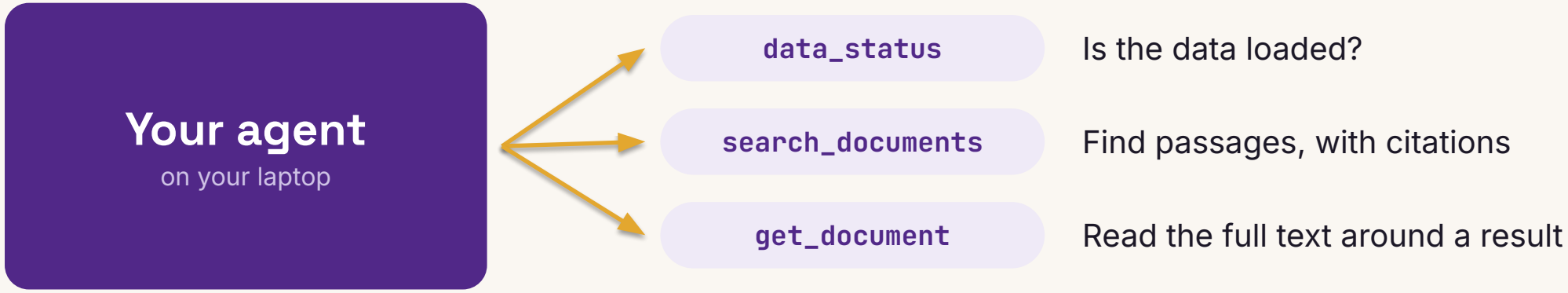
Your agent sets it up, then answers with the tools

1 · Set it up

Installs the project, loads the documents, runs rag-check and tells you what passed.

2 · Answer questions

Searches the documents with the three tools below, and cites what it found.



CHECKPOINT 2

Can your agent see the tools?

✓ **Data loaded**

data_status reports 1,847 records

→

2 **Tools connected**

Your agent lists the three tools

→

3 **Answer supported**

The cited page actually says it

`What tools do you have, and is the data loaded?`

It should call `data_status` and report **1,847 records**, bm25 mode.

Not there? Start a new session, from the `RAG-Symposium` folder itself.

Still no tools? Type: **“Find the full path of the uv program on this computer and put that full path in place of "uv" in the rag server settings.”** Then start a new session.

“How many grant applications can one researcher submit to NIH in a year?”

1

Question

What you typed.

→

2

Tool call

What the model searched for, in its own words.

`search_documents(…)`

→

3

Retrieved passage

The page that came back, with its date.

`get_document(…)`

→

4

Answer

With a citation you can open and check.

`[record-id]`

CHECKPOINT 3

Does the cited page actually say it?

✓ **Data loaded**

data_status reports 1,847 records

→

✓ **Tools connected**

Your agent lists the three tools

→

3 **Answer supported**

The cited page actually says it

How to check

Ask a question: your own, or one from the list.

Then ask: “Show me the exact passage you cited, and which document and page it’s from.”

Open that page yourself. Does it say that?

QUESTIONS THESE DOCUMENTS CAN ANSWER

- Can NIH reviewers use ChatGPT to help write their critiques?
- Do I have to tell NSF if I used AI to write my grant proposal?
- Can federal education grant money pay for a chatbot that advises students?
- Should there be a law against making an AI clone of my voice without permission?

PART TWO

Your conversations are a collection too.

Everything from the first hour applies to them.

1

WHY

What's worth keeping

2

HOW

Getting them out

3

LESSONS

Why search summaries

4

YOUR TURN

Search your own sessions

What’s in a transcript that isn’t anywhere else

The decision, and why

The reasons you gave and the options you turned down. Notes usually keep only the answer.

What didn’t work

The approaches you tried first. They rarely make it into notes, and they save time next round.

The prompt that worked

The exact wording that finally got a good result, so you don’t have to rediscover it.

What the AI claimed

Its exact words and the date, so you can check later what you were told.

What you can do with them later

`What did we decide about the grading rubric, and why?`	Find a decision and the reasoning behind it
`Where did I leave off on the grant draft?`	Pick up where you stopped, weeks later
`Catch a new colleague up on how this got built.`	Hand the context to a person, or to a fresh AI session
`Did it tell me something different last month?`	Notice when the answers changed

What disappears by default

Web chats live on someone else’s server

You can read them, but you only hold a copy if you export one.

Temporary chats are never saved

Incognito-style chats are gone when you close them.

Some agents clean up after themselves

Claude Code deletes its local session files after 30 days unless you change a setting.

Work accounts can switch export off

On team and enterprise plans, an administrator decides whether you can export.

What an export looks like

conversations.json

```
"title": "Grant timeline",
"mapping": {
  "a1f…": {
    "parent": "9c2…",
    "children": ["e40…", "b77…"],
    "message": {
      "author": {"role": "user"},
      "content": {"parts": ["…"]}
```

One big file

Every conversation you ever had, in one data file.

Branches

Each time you edited a message or regenerated, the file keeps both versions.

Gaps

Where a tool ran, some exports leave a placeholder. Uploaded files often aren’t included.

Agent sessions are noisier

They include every command, error message and retry.

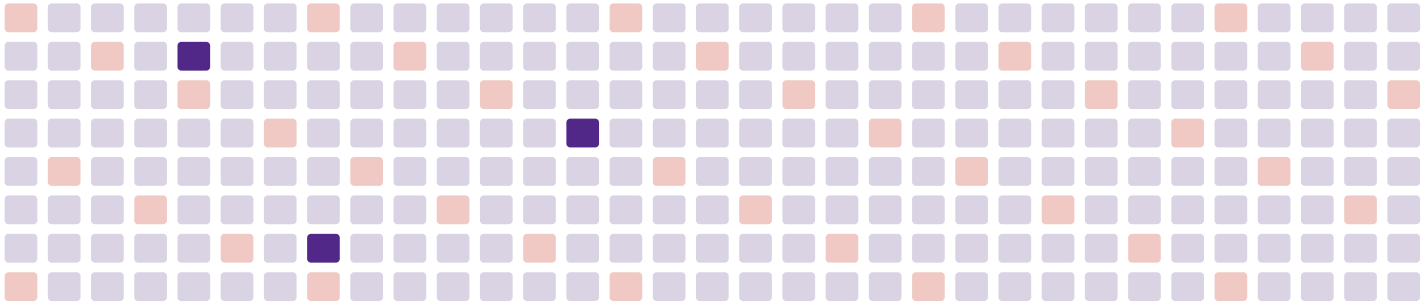
Most of a raw transcript is tool output

A WRITTEN NOTE



A few chunks, nearly all useful.

ONE LONG AGENT SESSION, SEARCHED RAW



Hundreds of chunks, sometimes over a thousand, mostly command output and error messages. The three that matter are in purple.

Keep the full transcript and index a short summary

In the index: one short summary per session

Title, date, your own messages and the agent's final reply.

`sessions/docs/`

every summary names its full transcript file

On disk: the full transcript

Not searched, but kept in full so you can check the summary.

`sessions/transcripts/`

YOU'LL RUN THIS NEXT

The tool you're about to run builds both layers for you.

Afterwards, to compare:

"Rebuild my sessions in raw mode, then ask the same question again."

Two ways to write the summary

WHAT RAG-SESSIONS DOES

Copy it out

Your own messages, word for word, plus the agent's final reply. No model is involved.

- ✓ It can't misquote you or change a decision.
- ✗ It can miss a decision made mid-session, and it reads rough.

THE OTHER WAY

Have a model write it

A model reads the session and writes a paragraph about it.

- ✓ It reads well and usually picks out the decision.
- ✗ It's the model's claim. It can drop the decision, turn "we decided not to" into "we decided to," or turn "maybe later" into "done."

Either way Keep the full transcript, name it in the summary, and check a few summaries against the original, the way you checked a citation.

Clean before you index

WHAT ENDS UP IN A TRANSCRIPT

- ✕ Passwords and keys you pasted to get something working
- ✕ Files a tool read, including settings files with secrets in them
- ✕ Student, patient or personnel details you pasted in
- ✕ Other people’s names, emails and messages

BEFORE IT BECOMES SEARCHABLE

- ✓ rag-sessions blanks out anything shaped like a password, key or card number. It only catches the obvious ones.
- ✓ Keep it on your own machine.
- ✓ Think about who else could search it, and what their agent would send to its model.

rag-sessions turns your sessions into documents

1 Find

Reads the session files your coding agent keeps on this laptop, or a chatbot export you point it at.

`read only`

→

2 Convert

Writes one document per session, labeled with its title, date and the tool it came from.

`sessions/docs/`

→

3 Redact

Replaces anything shaped like an API key, token or card number.

`[REDACTED]`

→

4 Load

Publishes them as a second collection that the same three search tools can read.

`data/sessions.jsonl`

THE DEFAULT: WHAT YOU'LL RUN

`--mode summary` Searches one short page per session, in your own words. Keeps the full transcript beside it and names the file.

OPTIONAL

`--mode raw` Searches every message, tool call and tool output.

It uses no model and nothing leaves your laptop. It reads Claude Code, Codex, OpenCode and Gemini CLI sessions, plus ChatGPT and Claude exports.

What leaves your laptop?

STAYS HERE

The documents and the search

All on your laptop: your files, the index, the search and our embedding model.

LEAVES

Your question and what search returns

Every passage the search returns goes to whoever runs your model, including text you never read.

YOUR CALL

What to do about it

Check what your provider and your institution allow.

Leave out what shouldn’t be searchable.

Stay informed on data retention policies.

YOUR TURN

Search your own AI sessions

TELL YOUR AGENT

Turn my AI sessions on this laptop into a searchable collection.

It finds your sessions, blanks out passwords and keys, and writes a short summary of each, all on your laptop.

THEN START A NEW SESSION AND ASK

How many transcripts did we pull in?

Then ask your own: a decision, a prompt you wish you'd kept.

fallback: terminal

```
uv run rag-sessions
uv run rag-search "what went wrong during setup?" --data data/sessions.jsonl
# web chatbot export? uv run rag-sessions --export the-export.zip
```

Setting this up for your own documents

1. **Collect 20 to 50 real questions,** including a few with no answer.
2. **Start with BM25** and score it.
3. **Add embeddings** only if the score improves.
4. **Fix the documents:** dates, old versions marked, readable tables.
5. **Make it able to say "not found,"** and test that it does.
6. **Re-load when documents change.** Every wrong answer becomes a new test question.

Try it now

Tell your agent:

"Add this document to the collection," then give it a file or paste the text.

By hand: one text file per document in `corpus/docs/`, with a short header. Then:

terminal

```
uv run rag-ingest
uv run rag-eval
```

Plausible answers need checking.

Now you have a way to check.

github.com/Cambrian-Systems/RAG-Symposium

Grady Burns · Hayden Thatcher